

Milestone 4 Report

Smart MCQ Solver Challenge

Course: BSDA2001P – Introduction to Deep Learning and

Generative AI Name: Soumyadip Hazari

Student ID: 24f3004946

Objective

This milestone focuses on converting the Smart MCQ Solver Challenge into a supervised multiple-choice classification problem using transformer models. Instead of treating the problem as a semantic similarity or retrieval task, every question is represented as five possible input sequences corresponding to options A-E. A BERT-based Multiple Choice model is then used to assign a score to every option. To make fine-tuning computationally efficient, Low-Rank Adaptation (LoRA) is applied so that only a small number of adapter parameters are trained while the original BERT weights remain frozen. Finally, the Hugging Face Trainer API is used to perform a small fine-tuning experiment and the fine-tuned model is evaluated by converting logits into answer probabilities.

Work Performed

1. Import Required Libraries and Loading Dataset—

The first step of the milestone contains the importing necessary libraries required for the data processing, tokenization, transformer models, GPU computation and fine tuning for which pandas, numpy, torch, Transformers, datasets, peft(LoRA) and Trainer API were imported.

Later By using pandas we loaded train.csv, test.csv and sample_submission.csv and also created a copy of each datasets to make sure that original datasets never get changed

2. Label Encoding

According to the milestone problems the second step is to encode the answers columns as models cannot train directly on categorical labels such as A,B,C,D,E so what we did is to convert it to the numerical labels using A → 0, B → 1, C → 2, D → 3, E → 4, and a new column known as "label" is created and added to the dataframe.

And to Ensure the conversion we tried to finding the numeric label for the row at index 150 which we get as 2

```
label_mapping = {"A": 0, "B": 1, "C": 2, "D": 3, "E": 4}
train_cc["label"] = train_cc["answer"].map(label_mapping)
```

```
train_cc.loc[150, ["answer", "label"]]
```

3. Prompt and Option Formatting

So after the label encoding we created a format where we combined every answer option with the question prompt using Prompt + [SEP] + Option for example Prompt → what is ...? → [SEP] → Option B and this formatting is repeated for all five options. We have done this step cause unlike the traditional classification where only the question is given to the model, a Multiple Choice transformer evaluates every answer choice individually.

The separator token [SEP] allows BERT to distinguish between the question and also the candidates answer while processing both together.

And to ensure the formatting we tried to find out the exact character length of `str(prompt) + " [SEP] " + str(option_B)` formatted input string which we get 405

4. Multiple Choice Tokenization

Now after formatting we tokenized the formatted inputs using `bert-base-uncased` with `padding = max_length`, `truncation = True` and `max_length = 128`. The tokenizer converted text into `input_ids`, `attention_mask` for each answer choice. Transformers cannot process raw text directly, they require numerical token ids so each question becomes Prompt + Option therefore every question generates five tokenized sequences. After reshaping for a multiple-choice model, the final `input_ids` tensor has shape: `[1, 5, 128]` so we tried to find the value of the second dimension which we get as 5.

5. Batch Tokenization

So instead of tokenizing only one question we later processed the first sixteen question as deep learning models are trained in batches rather than one sample at a time. Batch processing improves GPU utilization and significantly speeds up training. This results tensor shape of `[16,5,128]` so we tried to find the token positions which we get 10240

6. Multiple Choice Model

For this we loaded `AutoModelForMultipleChoice` using `bert-base-uncased` and performed a forward pass on one question. Ordinary Bert classification which predicts only one label directly we used Multiple choice model which compares all five options simultaneously so it produces one score (logit) for every answer choice. So the output logits tensor has shape `[1,5]` so we tried to find out the no. of logits are produced for one question which we get 5.

7. Supervised Loss Computation

The encoded label was supplied together with the tokenized inputs and the model automatically computed Cross-Entropy Loss as the loss function measures how different the model's prediction is from the correct answer, during training this loss

is minimized using gradient descent. The dimension of the loss function is 0.

8. Applying LoRA

Instead of updating every BERT parameter, LoRA adapters were inserted into Query, Value projection layers with configuration used $r = 8$, $\text{lora_alpha} = 16$, $\text{dropout} = 0.1$. Full fine-tuning updates every parameter inside BERT, requiring large GPU memory while LoRA freezes almost all original parameters and trains only small adapter matrices which reduces GPU memory, training time and number of trainable parameters. And then we counted the no. of parameters that is trainable for which we got 295681.

9. Preparing Hugging Face Dataset and Tiny Fine-Tuning

The first 100 training samples were converted into a Hugging Face Dataset containing `input_ids`, `attention_mask`, labels and then a small fine-tuning experiment was then performed using the first 32 samples. We kept the training settings Batch Size = 4, Gradient Accumulation = 1, Max Steps = 4. After the successful completion of the trainer we tried to find the global step which we get 4.

10. Inference after Fine-Tuning

After fine-tuning, the model was switched into evaluation mode. The first question was passed through the model. The output logits were converted into probabilities using Softmax. As logits themselves are not probabilities so softmax converts them into values between 0 and 1 whose sum equals to 1 which allows the probabilities of every answer option to be interpreted. So we tried to find the probability for the option E which we get 0.1977.

Key Results

Experiment	Result
Label Encoding (Row 150)	2
Prompt + Option B Character Length	405
Single MCQ Tokenization (Second Dimension)	5
Batch Token Positions	10240
Multiple Choice Logits	5
Loss Tensor Dimensions	0
LoRA Trainable Parameters	295681

Tokenized Choices per Example	5
Trainer Global Step	4
Probability of Option E	0.1977

Conclusion

This milestone successfully demonstrated how a multiple-choice transformer model can be fine-tuned efficiently using LoRA. We transformed raw MCQ data into model-ready inputs, encoded categorical labels, tokenized every answer choice, and trained a BERT-based Multiple Choice model using the Hugging Face Trainer. Unlike the previous milestones that focused on semantic similarity, retrieval, and RAG, this milestone introduced supervised fine-tuning, parameter-efficient learning, and probability-based inference. These techniques form the foundation for building transformer models that can directly learn to rank and predict the correct answer choices for the Smart MCQ Solver Challenge.